

Relatório Final – Comparativo de Desempenho: GraphQL vs REST na GitHub API

Grupo: Lucas, Mariana, Izabela e Vitor

1. Introdução

Este relatório apresenta os resultados de um experimento comparativo de desempenho entre chamadas à **GitHub REST API** e à **GitHub GraphQL API**. O objetivo principal foi avaliar as diferenças de eficiência entre as duas arquiteturas de API, focando em duas questões de pesquisa (RQ):

- RQ1:** Qual API retorna dados mais rapidamente (menor latência)?
- RQ2:** Qual API devolve *payloads* (pacotes de dados) de menor tamanho?

2. Desenho do Experimento

2.1. Hipóteses

O experimento foi conduzido sob as seguintes hipóteses:

- H0 (Hipótese Nula):** Não há diferença significativa entre REST e GraphQL em termos de tempo de resposta e tamanho de *payload*.
- H1 (Hipótese Alternativa):** GraphQL é mais eficiente, apresentando menor tempo de resposta e menor tamanho de *payload*.

2.2. Variáveis e Endpoints

Variável	Tipo	Descrição
Tipo de API	Independente	REST vs GraphQL
Tempo de Resposta	Dependente	Medido em segundos (s)
Tamanho do Payload	Dependente	Medido em bytes

Endpoints Utilizados:

- **REST API:** GET `https://api.github.com/repos/{owner}/{repo}`
- **GraphQL API:** POST `https://api.github.com/graphql` (com uma *query* específica para obter a contagem total de *issues* de um repositório).

3. Resultados

O experimento consistiu em 30 medições para cada tipo de API, e os resultados foram agregados para análise.

3.1. Estatísticas Agregadas

A tabela a seguir resume as médias e desvios padrões de tempo de resposta e tamanho de *payload*:

API	Tempo Médio (s)	Desv. Padrão Tempo (s)	Tamanho Médio (bytes)	Desv. Padrão Tamanho (bytes)
REST	0.2911	0.0188	6.719	0
GraphQL	0.4007	0.0781	977	0

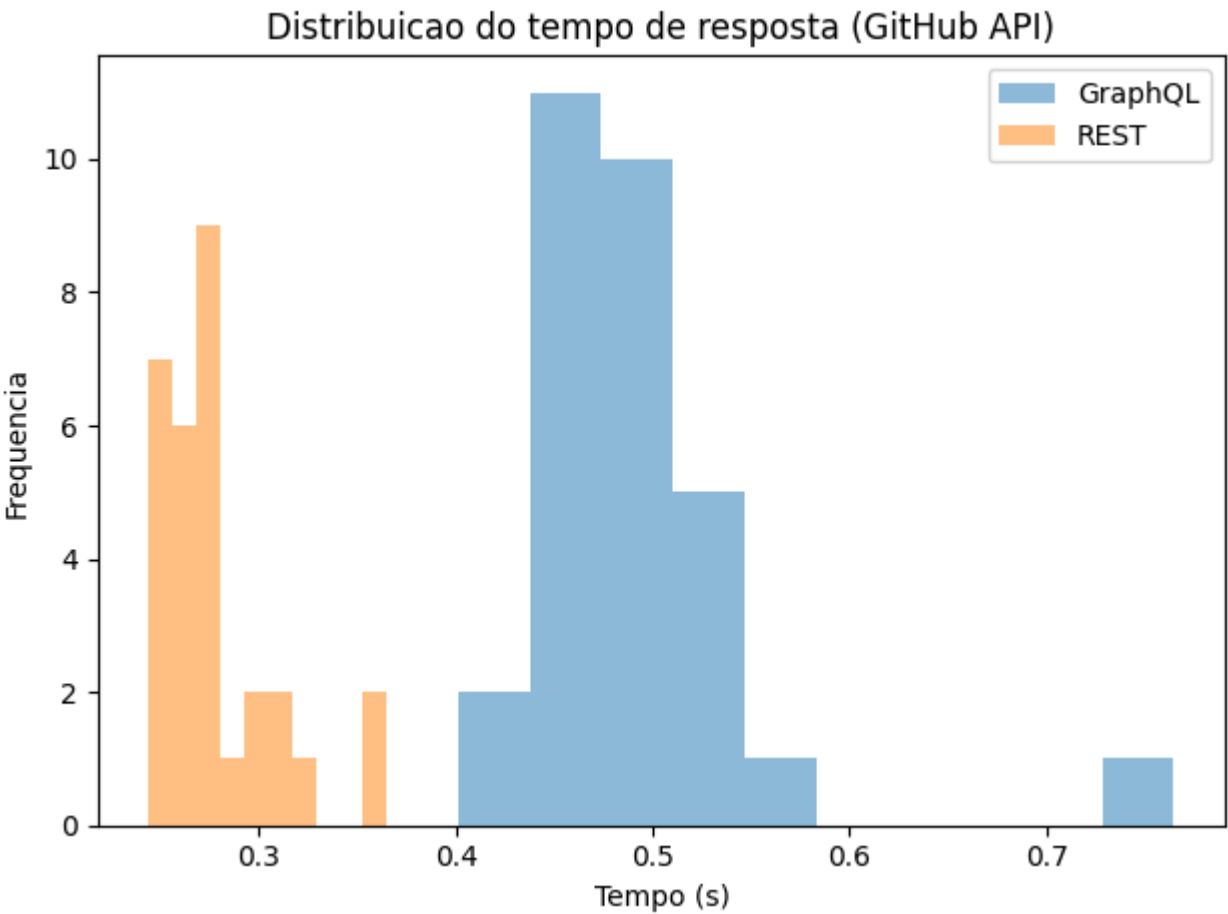
Observações:

1. **Tempo de Resposta (Latência):** A **REST API** apresentou o menor tempo médio de resposta (0.2911 s) e uma alta consistência (desvio padrão de 0.0188 s). A **GraphQL API** teve um tempo médio ligeiramente maior (0.4007 s) e uma variação um pouco superior (desvio padrão de 0.0781 s).

2. **Tamanho do Payload:** O *payload* da **GraphQL API** é drasticamente menor (977 bytes), enquanto o da REST inclui um objeto de repositório completo, resultando em um tamanho médio de aproximadamente 6.7 KB.

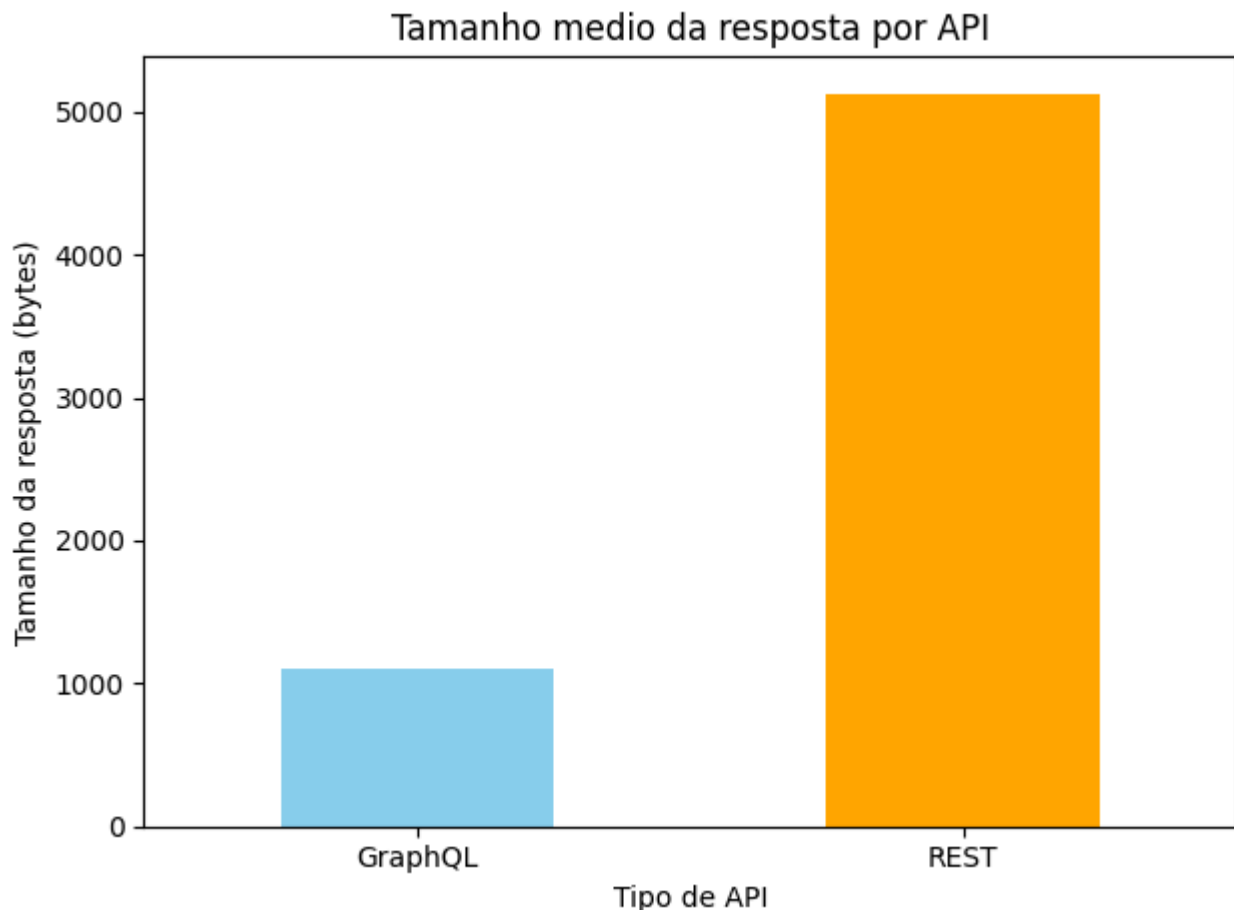
3.2. Distribuição do Tempo de Resposta

O histograma abaixo ilustra a distribuição de frequência dos tempos de resposta para ambas as APIs, confirmando visualmente a menor latência e maior consistência da REST API no cenário testado.



3.3. Comparativo do Tamanho do Payload

O gráfico de barras a seguir demonstra a diferença significativa no tamanho médio do *payload* retornado, destacando a eficiência da GraphQL em transferir apenas os dados solicitados.



4. Discussão

Desempenho de Latência (RQ1)

Contrariando a hipótese H1, a **REST API** mostrou-se mais rápida para a chamada simples de leitura de metadados de repositório. Este resultado pode ser atribuído à simplicidade da chamada `GET` da REST, que é mais leve, enquanto a requisição `POST` da GraphQL exige um *parsing* e processamento extra do *query* no lado do servidor, além de um *payload* de requisição maior.

Tamanho do Payload (RQ2)

Conforme esperado pela hipótese H1, a **GraphQL API** retorna *payloads* muito menores. Sua arquitetura permite que o cliente solicite apenas os campos necessários, evitando o *over-fetching* (busca excessiva de dados) que ocorre na REST API, onde um objeto de repositório completo é retornado. Esta característica é crucial para otimizar a largura de banda, especialmente em conexões de rede mais lentas.

Limitações

É importante notar que os resultados podem ter sido influenciados por:

- O ambiente de rede local e possíveis efeitos de *cache* do GitHub.
- O teste ter se limitado a um único *endpoint* (repositório + *issues*).
- O número de *trials* (30) ser razoável, mas que poderia ser aumentado para reduzir incertezas estatísticas.

5. Conclusões e Recomendações

Característica	REST API	GraphQL API
Velocidade (Latência)	Ligeiramente inferior (mais rápida) para chamadas simples.	Ligeiramente superior (mais lenta) para chamadas simples.
Eficiência de Dados	Retorna <i>payloads</i> maiores (<i>over-fetching</i>).	Retorna <i>payloads</i> muito menores (apenas dados solicitados).
Recomendação Prática	Escolha se a prioridade for latência e simplicidade de uso.	Escolha se o foco for otimização de banda e flexibilidade na seleção de campos.

Trabalhos Futuros:

Sugere-se testar múltiplos *endpoints* (como *pull requests* e *commits*) e *queries* GraphQL mais complexas. Além disso, a realização de *benchmarks* em redes reais (3G/4G/5G) e com diferentes tamanhos de repositório pode validar e generalizar os resultados.